

Explicación de Capas — Arquitectura MVVM

Sistema de Gestión de Órdenes y Entregas · NativeScript + ASP.NET Core 8

Capa 1 — View (interfaz de usuario)

La capa View está compuesta por los archivos XML de NativeScript. Es la única parte del sistema que el usuario ve directamente. Su responsabilidad es mostrar información y capturar acciones del usuario, como tocar un botón o ingresar texto en un formulario.

Esta capa no contiene lógica de negocio ni accede a servicios directamente. Toda interacción se conecta al ViewModel mediante el mecanismo de data binding de NativeScript, que actualiza la pantalla automáticamente cuando los datos cambian.

Las pantallas que forman esta capa son:

- home-page.xml — lista principal de entregas
- delivery-create-page.xml — formulario para registrar una nueva entrega.
- delivery-detail-page.xml — detalle de una entrega con estado y conductor.
- edit-delivery-page.xml — edición de entrega y cambio de estado.
- login-page.xml — pantalla de autenticación del usuario.
- create-client-page.xml — registro de nuevos clientes.
- create-driver-page.xml — registro de conductores con carga de foto.

Capa 2 — ViewModel (lógica de presentación)

El ViewModel actúa como intermediario entre la View y los servicios. Cada pantalla tiene su propio ViewModel escrito en TypeScript que extiende la clase Observable de NativeScript.

Esta capa es responsable de manejar el estado de la interfaz: qué datos se muestran, si hay una carga en progreso, si la lista está vacía, o si ocurrió un error. También ejecuta las validaciones de los formularios antes de enviar datos al backend.

Los ViewModels del proyecto son:

- home-view-model.ts — carga la lista de entregas, maneja la búsqueda y los estados de carga y lista vacía.
- create-delivery-view-model.ts — valida el formulario de nueva entrega y llama al servicio para crearla.
- delivery-detail-view-model.ts — obtiene el detalle de una entrega incluyendo información del conductor.
- edit-delivery-view-model.ts — gestiona el cambio de estado de una entrega.
- login-view-model.ts — maneja el flujo de autenticación y almacena el token JWT en sesión.
- create-driver-view-model.ts — gestiona el registro de conductores con selección y carga de foto.

Capa 3 — Services (acceso a datos)

Los servicios son clases TypeScript con una única responsabilidad cada una. Los ViewModels los consumen para obtener o enviar datos, sin importar cómo o de dónde provienen esos datos. Esta separación permite cambiar la fuente de datos sin modificar la lógica de presentación.

Los servicios que componen esta capa son:

- `delivery.service.ts` — realiza las operaciones CRUD de entregas contra la API REST del backend.
- `auth.service.ts` — gestiona el login del usuario y el almacenamiento del token JWT.
- `driver.service.ts` — consulta y registra conductores, incluyendo la carga de imágenes de perfil.
- `client.service.ts` — registro y consulta de clientes.
- `sqlite.service.ts` — proporciona persistencia local mediante SQLite para almacenar entregas en caché y permitir acceso sin conexión.
- `firebase.service.ts` — inicializa Firebase en la app y escucha mensajes push entrantes.
- `notification.service.ts` — registra el token FCM del dispositivo y gestiona el envío de notificaciones de estado.
- `connectivity.service.ts` — detecta el estado de la conexión a internet para decidir si consultar la API o la base de datos local.
- `geocoding.service.ts` — resuelve coordenadas o nombres de lugar para los campos de origen y destino.

Capa 4 — Backend REST (ASP.NET Core 8)

El backend es una API REST desarrollada en C# con ASP.NET Core 8. Recibe las solicitudes HTTP del frontend, aplica la lógica de negocio y persiste los datos en SQL Server mediante Entity Framework Core. Toda la API está protegida con autenticación JWT; algunas operaciones requieren además el rol de administrador.

La capa de backend se organiza internamente en tres niveles:

Controllers

Son los puntos de entrada de la API. Cada controlador agrupa los endpoints de un módulo y delega el trabajo al servicio correspondiente. Los controladores del proyecto son:

- `DeliveriesController` — expone los endpoints de entregas: listar, obtener por ID, crear, actualizar, cambiar estado y eliminar.
- `AuthController` — gestiona el inicio de sesión y la generación del token JWT.
- `DriversController` — administra el registro de conductores y la subida de fotos al servidor.
- `NotificationsController` — recibe y registra los tokens FCM de los dispositivos para enviar notificaciones push.

Services

Contienen la lógica de negocio. Cada servicio tiene una interfaz (por ejemplo, `IDeliveryService`) y su implementación concreta (`DeliveryService`). Esto permite desacoplar el controlador de la implementación y facilitar las pruebas unitarias.

Repositories

Son responsables exclusivamente del acceso a la base de datos. Implementan interfaces como `IDeliveryRepository` y utilizan el `DbContext` de Entity Framework Core para ejecutar consultas sobre SQL Server. Esta separación evita que la lógica de negocio tenga conocimiento directo de la base de datos.

Models

Se dividen en dos grupos. Las entidades (carpeta `Entities`) representan las tablas de la base de datos: `Delivery`, `Driver`, `Client`, `User` y `Notification`. Los DTOs (carpeta `DTOs`) son objetos de transferencia usados para recibir datos del frontend (`Request`) o enviarle respuestas (`Response`), sin exponer directamente la estructura interna de la base de datos.

Capa 5 — Infraestructura externa

Es la capa más externa del sistema. Contiene los servicios de terceros y el almacenamiento que tanto el frontend como el backend consumen:

- SQL Server — base de datos relacional donde se persisten todas las entidades del sistema, gestionada a través de SQL Server Management Studio.
- Firebase Authentication — servicio de autenticación de usuarios utilizado como complemento al sistema de JWT del backend.
- Firebase Cloud Messaging (FCM) — plataforma de notificaciones push. El backend utiliza el SDK `FirebaseAdmin` para enviar notificaciones cuando el estado de una entrega cambia.
- Almacenamiento de archivos (`wwwroot/uploads`) — el servidor almacena en disco las fotos de perfil de los conductores y expone su URL pública en el campo `photoUrl` del `DriverDetailDto`.